

Polytech Dijon

Computer Science

3D Mesh

Rapport mi-parcours

Réalisé par :
CLOIX Mattys
RAFFIN Alexandre

Supervisé par :
FOUGEROLLE Yohan

13 octobre 2026

Table des matières

1 Introduction et Objectifs **2**
1.1 Introduction 2
1.2 Objectifs 2

2 Planification **2**

3 Architecture Logicielle et UML **3**

4 État d'avancement du projet **6**

5 Analyse des Risques et Plans B **6**
5.1 Problèmes 6
5.2 Solutions 6

1 Introduction et Objectifs

1.1 Introduction

Ce projet de Computer Science a pour objectif de développer un éditeur de maillages 3D interactif en C++. Il aura pour but l'étude des structures de données, de la programmation orientée objet et de la représentation des objets géométriques en trois dimensions.

L'application devra permettre d'importer des modèles 3D à partir de fichiers géométriques, avec par exemple le format `.obj`, puis de les représenter graphiquement à l'aide d'OpenGL. Pour assurer une gestion efficace des relations entre les sommets, les arêtes et les faces du maillage, nous utiliserons la structure de données Half-Edge.

Le travail est a réalisé en binôme, avec pour objectif final de réussir à obtenir une application fonctionnelle, structurée et avec un compte rendu présenté au format LaTex.

1.2 Objectifs

L'objectif principal est de mettre en place un éditeur capable de visualiser et de modifier un maillage 3D tout en préservant la cohérence de sa structure. Plusieurs opérations topologiques sont ainsi prévues, comme la subdivision d'une arête, le retournement d'une arête ou encore la suppression de sommets et de faces suivant le besoin.

De plus, il faudra mettre en pratique des principes de la programmation orientée objet et de la conception logicielle. Nous devrons, par conséquent, concevoir une architecture adaptée, organiser les différentes fonctionnalités et veiller à la fiabilité de l'application.

2 Planification

Afin d'organiser le travail sur l'ensemble du projet, nous avons établi un diagramme de Gantt présentant les différentes étapes prévues pour notre développement. Ainsi, nous avons noté leurs ordres de réalisation ainsi que leur répartition dans le temps. De ce fait, cette planification nous permet de suivre l'avancement du projet et de répartir le travail entre les deux membres du binôme tout en connaissant la chronologie du projet.

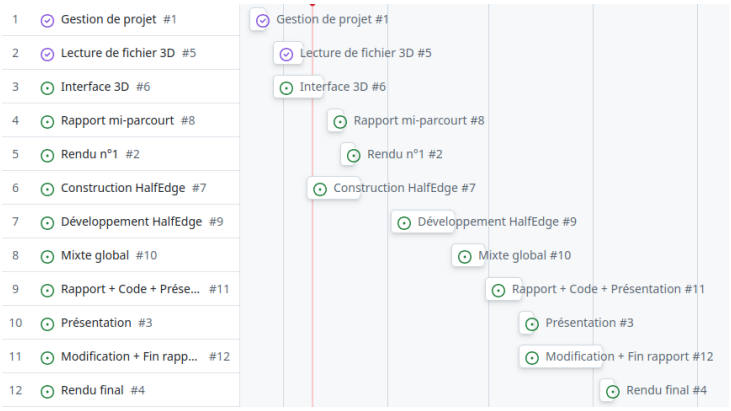


FIGURE 1 – Diagramme de Gantt

En outre, en complément de cette planification, nous avons créé un dépôt GitHub comme demandé dans le sujet afin de centraliser tout le travail demandé tel que le code source du projet, ce qui permet de faciliter le travail en binôme. Ceci nous permet de suivre les modifications apportées au code, de conserver un historique des versions et de coordonner le développement entre les deux membres du groupe.

3 Architecture Logicielle et UML

Avant de débuter la phase de programmation et d'écriture du code, il est essentiel de concevoir l'architecture du projet. C'est l'un des principes fondamentaux de la programmation orientée objet. Pour ce faire, l'outil le plus répandu et efficace est le diagramme de classes UML, qui permet de visualiser la structure des classes ainsi que leurs interactions.

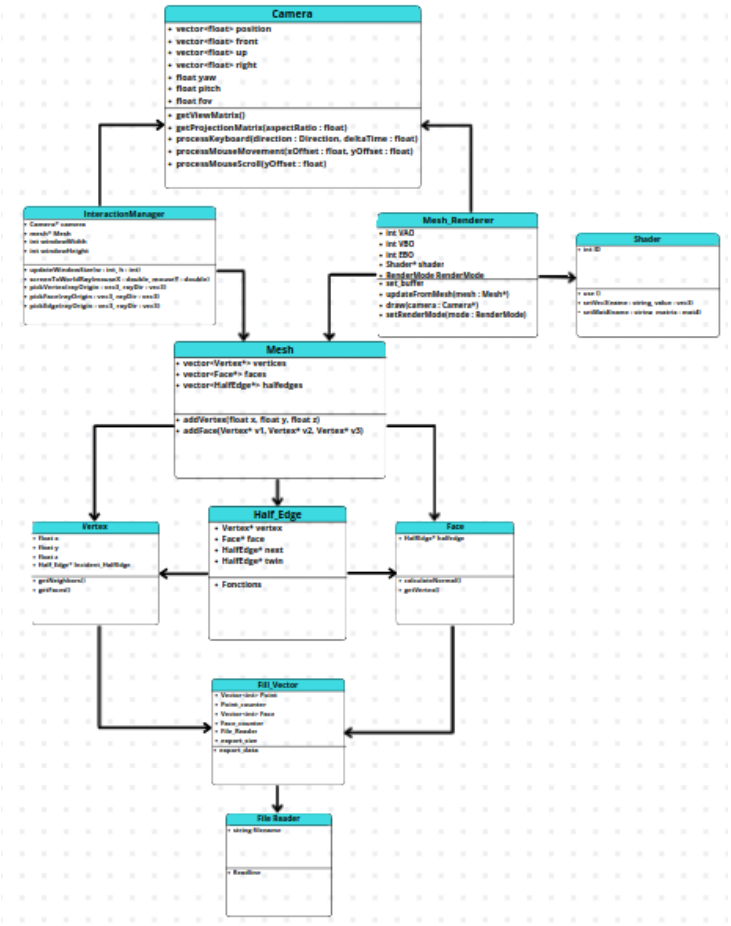


FIGURE 2 – Diagramme UML

Ci-dessus, nous pouvons voir notre diagramme UML avant programmation. Il est évident que celui-ci est voué à être un peu modifié, car nous pourrons être confrontés à des problèmes futurs qui devront être résolus à l'aide de fonctions ou de classes auxquelles nous n'avions pas encore pensé.

Notre diagramme UML a été structuré en deux parties principales. La première concerne la structure Half-Edge, regroupant l'ensemble de la chaîne de traitement, du chargement des données du fichier .obj jusqu'à la construction des vecteurs de demi-arêtes.
La seconde partie est dédiée à OpenGL et au rendu graphique. Cette dernière est la plus susceptible d'évoluer, car elle repose sur des concepts que nous maîtrisons moins pour le moment, notamment la gestion de la caméra et des shaders, indispensables sous OpenGL.

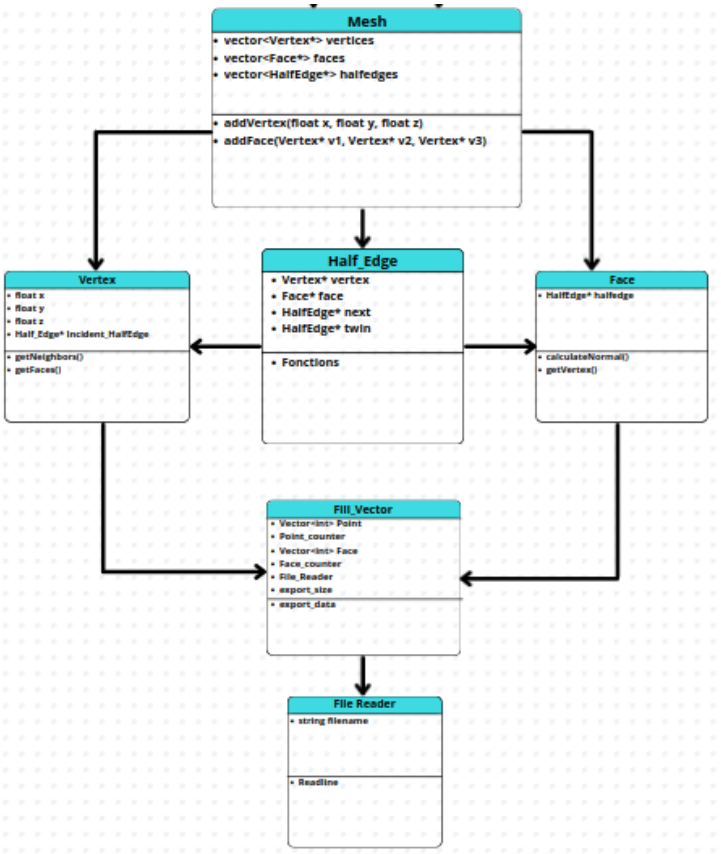


FIGURE 3 – UML de la partie Half edge

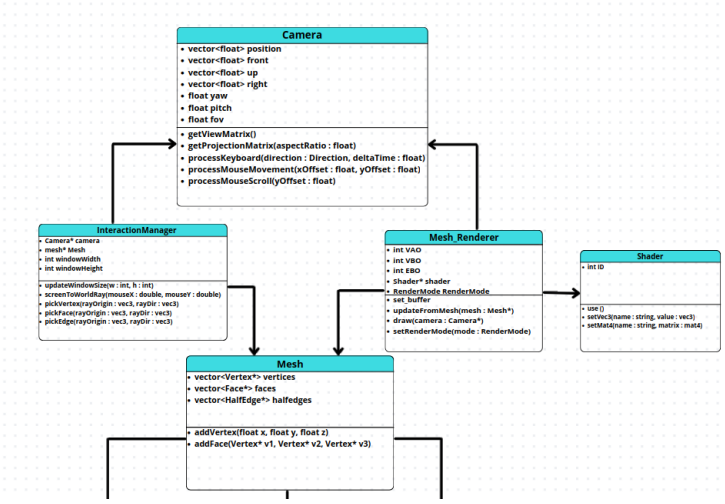


FIGURE 4 – UML de la partie OpenGL

4 État d'avancement du projet

5 Analyse des Risques et Plans B

5.1 Problèmes

Plusieurs difficultés peuvent apparaître au cours du projet :
La structure Half-Edge peut être complexe à mettre en place et à maintenir cohérente.
L'utilisation d'OpenGL peut également poser des problèmes lors de l'affichage des maillages 3D.
Enfin, le développement en binôme peut entraîner des conflits de code ou des retards dans la réalisation des fonctionnalités.

5.2 Solutions

Il y a bien sûr différentes solutions pour éviter ces problèmes :
Nous pouvons réaliser des tests réguliers afin de vérifier le bon fonctionnement du programme et en cas de problème avec certaines fonctionnalités, nous privilégierons une version plus simple permettant de conserver les fonctions essentielles du logiciel.
L'utilisation de GitHub nous permettra de partager le code et de suivre les modifications effectuées par chaque membre du binôme.
Enfin, pour les retards il suffit de se mettre au travail !